

漏洞报告 VULN-01：弱口令与密码爆破

漏洞类型：弱口令与密码爆破

测试小组：天亮了吗

被测小组：天亮了吗

测试时间：2026-08-20

测试环境：本机 localhost / 127.0.0.1:5001

测试账号：课程测试账号，未使用真实个人密码

1. 基本信息

本报告针对阶段二任务1，仅在授权的本机课程靶场中完成。

2. 漏洞位置

URL/接口： /login、/register、/api/login、/api/register

请求方法： GET/POST/DELETE（按接口实际方法）

关键参数： username、password、filename、file、csrf_token（按任务适用）

所属功能： 认证

3. 正常请求

先使用正常账号和合法参数完成对应功能，再仅修改一个测试参数。请求记录和页面证据见附件目录。

正常请求示例：在本机浏览器访问对应页面，提交课程测试账号或合法文件；不包含真实密码、Cookie 或 Token。

4. 漏洞复现步骤

- 启动本机网盘并登录课程测试账号。
- 按任务说明构造单一无害测试参数。
- 发送请求并记录状态码、页面、文件或会话变化。
- 使用修复后的同一参数再次验证。

5. 攻击效果或异常现象

历史账号存在 dio/dio、zxrnb/123456 等弱密码；修复前可直接登录。修复后连续5次错误返回401并锁定，弱密码注册返回400。

6. 漏洞原因分析

gets.py 的 _login、_register 缺少密码策略、失败计数和安全日志。

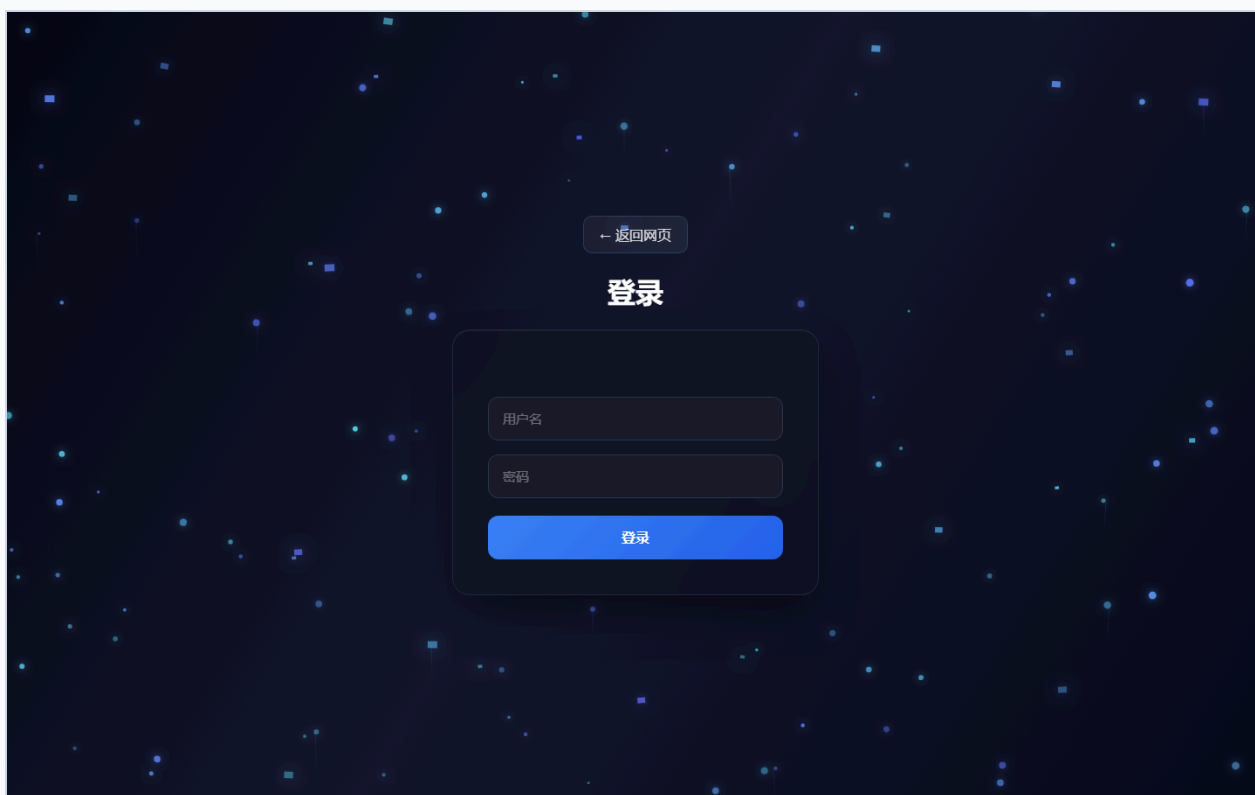
7. 影响范围

影响范围限于本机课程靶场及测试账号；不得外推为公网系统影响。具体后果取决于接口是否需要登录及资源归属。

8. 修复建议

增加8位及字母数字密码策略、弱口令黑名单、5次/5分钟锁定、统一错误提示、登录日志；历史明文密码成功登录后升级哈希。

9. 附件清单



[← 返回网页](#)

注册

密码长度至少为8位

weak-proof

密码

确认密码

注册

[← 返回网页](#)

登录

用户名或密码错误

pyknb

密码

登录

[← 返回网页](#)

登录

用户名或密码错误

dio

密码

登录

修复代码截图（实际源文件与行号）

任务 01 修复代码证据

gets.py:105-125

```
105 state = login_failures.setdefault(key, {'count': 0, 'locked_until': 0})
106 if state['locked_until'] and state['locked_until'] < time.time():
107     state['count'] = 0
108     state['locked_until'] = 0
109     state['count'] += 1
110 if state['count'] > LOGIN_MAX_FAILURES:
111     state['locked_until'] = time.time() + LOGIN_LOCK_SECONDS
112     security_logger.warning("login_failed username=%s ip=%s reason=%s count=%s", username, client_ip(), reason, state['count'])
113
114
115 def login_locked(username):
116     state = login_failures.get(login_key(username))
117     return bool(state and state['locked_until'] > time.time())
118
119
120 def clear_login_failures(username):
121     login_failures.pop(login_key(username), None)
122
123
124 def get_db_connection():
125     return pymysql.connect(**db_config)
```

gets.py:163-215

```
163 def login_page():
164     csrf_token()
165     return render_template("login.html")
166
167
168 @app.post("/login")
169 @csrf_protected
170 def login_post():
171     username = request.form.get("username", "").strip()
172     password = request.form.get("password", "")
173     result = _login(username, password, as_api=False)
174     if isinstance(result, tuple) and len(result) == 2 and isinstance(result[1], int) and result[1] >= 400:
175         return render_template("login.html", error=result[0], username=username, result[1])
176     return result
177
178
179 def _login(username, password, as_api=True):
180     if not username or not password:
181         response = {'error': "请输入用户名和密码"}
182         return jsonify(response), 400 if as_api else ("请输入用户名和密码", 400)
183     if login_locked(username):
184         response = {'error': "用户名或密码错误"}
185         return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
186     conn = get_db_connection()
187     try:
188         with conn.cursor() as cursor:
189             cursor.execute("SELECT username, password FROM users WHERE username=%s", (username,))
190             user = cursor.fetchone()
191         finally:
192             conn.close()
193     valid = bool(user) and (
194         check_password_hash(user[1], password)
195         if user and user[1].startswith("{pbkdf2}", "bcrypt:"))
196     else bool(user) and user[1] == password
197 )
198 if not valid:
199     record_login_failure(username)
200     response = {'error': "用户名或密码错误"}
201     return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
202 clear_login_failures(username)
203 session.clear()
204 csrf_token()
205 session["username"] = username
206 if user and not user[1].startswith("{pbkdf2}", "bcrypt:"):
207     conn = get_db_connection()
208     try:
209         with conn.cursor() as cursor:
210             cursor.execute("UPDATE users SET password=%s WHERE username=%s", (generate_password_hash(password), username))
211         conn.commit()
212     finally:
213         conn.close()
214     security_logger.info("login_success username=%s ip=%s", username, client_ip())
215     if as_api:
```

gets.py:267-302

```
267 @csrf_protected
268 def register_post():
269     data = request.form
270     result = _register(data.get("username", "").strip(), data.get("password", ""), data.get("confirm_password", ""))
271     if result[1] != 200:
272         payload = result[0].get_json(silent=True) if hasattr(result[0], "get_json") else 0
273         return render_template("register.html", error=(payload or 0).get("error", "注册失败"), username=data.get("username", ""), result[1])
274     return redirect("/")
275
276
277 def _register(username, password, confirm_password):
278     if not username or not password:
279         return jsonify({'error': "用户名和密码不能为空"}), 400
280     if password != confirm_password:
281         return jsonify({'error': "两次密码不一致"}), 400
282     policy_error = password_error(username, password)
283     if policy_error:
284         return jsonify({'error': policy_error}), 400
285     conn = get_db_connection()
286     try:
287         with conn.cursor() as cursor:
288             cursor.execute("SELECT 1 FROM users WHERE username=%s", (username,))
289             if cursor.fetchone():
290                 return jsonify({'error': "用户名已存在"}), 409
291             cursor.execute(
292                 "INSERT INTO users(username, password) VALUES(%s, %s)",
293                 (username, generate_password_hash(password)),
294             )
295         conn.commit()
296     finally:
297         conn.close()
298     return jsonify({'success': True}), 200
299
300
301 @app.post("/api/register")
302 @csrf_protected
```